Four-way comparison: coral runs of 2026-10-20

Four 20-real-attempt CORAL evolutions, one per branch, each on its own CloudLab host so that no two instances shared a node. The branches cross two choices: seeds (all seven reference policies, or S3-FIFO only) and scoring (baseline-normalised tanh z-score, or absolute throughput in ops/s). Only these four runs are compared here.

run	scoring	seeds	host	real / tune / crashed	wall	LLM cost	best score	its tput (ops/s)	max tput (ops/s)
all-seeds / tanh	tanh z-score	7 + no-op	pc306	20 / 8 / 1	0.94 h	$121	#4 346b8d9c 0.643	621.8	657.2 (#24)
one-seed / tanh	tanh z-score	S3-FIFO only	pc309	20 / 0 / 0	0.64 h	$38	#19 fefe1ef1 0.755	647.6	648.4 (#20)
all-seeds / max-throughput	absolute ops/s	7 + no-op	pc319	20 / 0 / 3	0.49 h	$37	#18 e8ecbcfc 679.3	679.3	679.3 (#18)
one-seed / max-throughput	absolute ops/s	S3-FIFO only	pc320	20 / 0 / 1	0.48 h	$28	#6 46ce09fd 540.8	540.8	540.8 (#6)

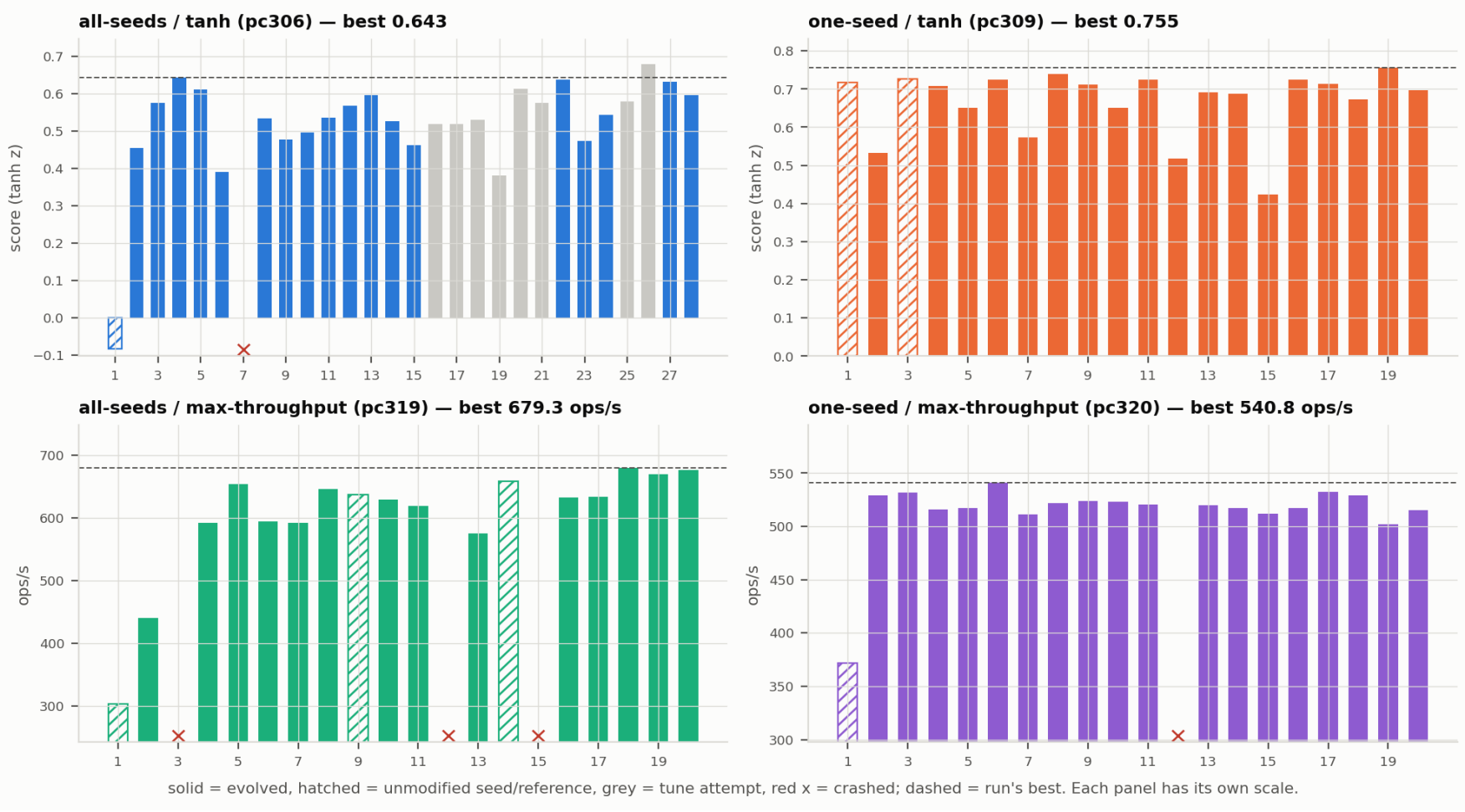
How to read this

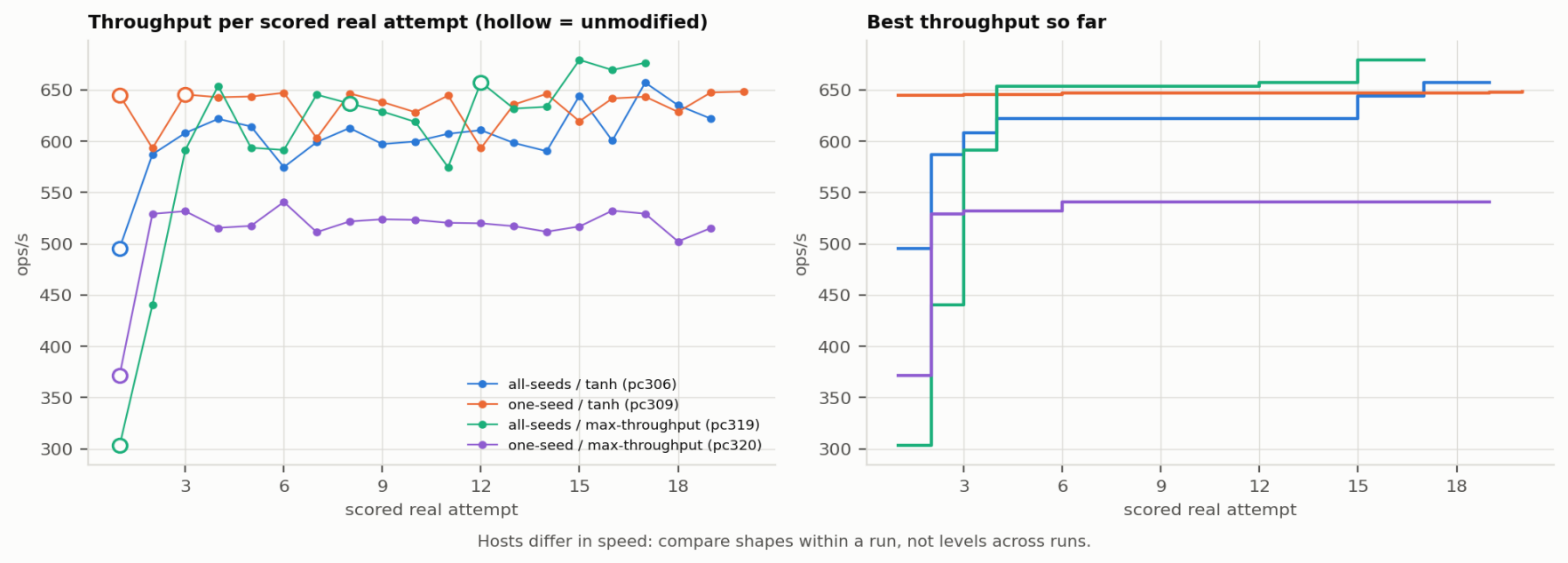
• The hosts are different machines. All four are 2-core, ~2 GB Xeons, but the same code runs at different speeds on them: unmodified S3-FIFO measured about 645 ops/s on pc309 and 637 on pc319, while evolved S3-FIFO variants sit around 520-540 on pc320. Throughput levels are therefore not comparable across runs; only the change within a run is.

• Scores are on two scales. tanh scores (in [-1, 1]) are relative to each host's own vanilla-kernel no-op baseline (n=8, 553 ± 109 ops/s on pc306 and 549 ± 101 on pc309; the std is ~20%, so z-scores are noisy). Absolute scores are raw ops/s.

• The first evaluation on a host reads low in some runs (pc319 first eval 303 vs about 640 for later unmodified policies; pc320 372 vs about 520-540 for its later attempts). I list such measurements but flag them; do not use them as a baseline.

• Differences of a few percent are within measurement noise (earlier runs saw 1-4% for identical code). No best policy was re-measured enough to separate it from its seed.


all-seeds / tanh (pc306) — best 0.643
score (tanh z)
one-seed / tanh (pc309) — best 0.755
score (tanh z)
all-seeds / max-throughput (pc319) — best 679.3 ops/s
ops/s
one-seed / max-throughput (pc320) — best 540.8 ops/s
ops/s
solid = evolved, hatched = unmodified seed/reference, grey = tune attempt, red x = crashed; dashed = run's best. Each panel has its own scale.



Dots are the same quantity (throughput) for every run, but each run has its own host.

Best evolved policy versus the seed it descends from (same host, same run)

run (host)	policy	ancestor seed	throughput ops/s	MiB read	MiB written	read I/Os	refaults	major faults	changed lines vs seed
all-seeds / tanh (pc306)	best evolved #4 346b8d9c	get_scan.c — heavily rewritten (closest seed only 67% similar); lineage root is attempt #1 (get_scan.c)	621.8	163	n/r	n/r	16960	n/r	493 added+removed (po
	unmodified get_scan.c, measured 1x (#1)	seed	495.7 (+25.4%)	134 (+21.8%)	n/r	n/r	10876 (+55.9%)	n/r	
	vanilla-kernel no-op baseline (grader calibration, n=8)	none	553.1 ± 109.2						
one-seed / tanh (pc309)	best evolved #19 fefe1ef1	s3fifo.c — closest seed by line similarity (99%)	647.6	166	n/r	n/r	17599	n/r	10 added+removed (poli
	unmodified s3fifo.c, measured 2x (#1 #3)	seed	645.3 (+0.4%)	171 (-2.5%)	n/r	n/r	18531 (-5.0%)	n/r	
	vanilla-kernel no-op baseline (grader calibration, n=8)	none	549.2 ± 100.9						
all-seeds / max-throughput (pc319)	best evolved #18 e8ecbcfc	mglru.c — closest seed by line similarity (98%)	679.3	218	0	11875	29693	23	34 added+removed (poli
	unmodified mglru.c, measured 1x (#14)	seed	657.5 (+3.3%)	196 (+11.7%)	0	11942	24545 (+21.0%)	22	
one-seed / max-throughput (pc320)	best evolved #6 46ce09fd	s3fifo.c — closest seed by line similarity (89%)	540.8	144	0	10337	12901	22	145 added+removed (po
	unmodified s3fifo.c, measured 1x (#1)	seed	371.9 (+45.4%)	166 (-13.3%)	35	9367	18210 (-29.2%)	22	

• n/r = not recorded: the tanh feedback only reports bytes read, refaults and throughput. Percentages are best vs the unmodified-seed mean of this run.

• Flagged baselines. The unmodified-seed rows for all-seeds/tanh (get_scan, #1: 495.7) and one-seed/max-throughput (S3-FIFO, #1: 371.9) are each the run's first evaluation on a host, and first evaluations read low (see the first page). The gains against those rows are inflated and should not be quoted as improvements. The cleaner comparisons are one-seed/tanh (S3-FIFO measured twice, 645.0 and 645.6, against a best of 647.6: +0.4%, inside noise) and all-seeds/max-throughput (MGLRU 657.6 against 679.3: +3.3%, one measurement each).

• A policy's best score is the maximum over 20 noisy attempts, so it is biased upward relative to its seed's single measurement.

all-seeds / tanh (pc306): best policy #4 346b8d9c — score 0.643, 621.8 ops/s

A rewrite in the spirit of get_scan.c: two lists with no per-folio map for scan pages. Folios added by a scan thread (the harness's scan_pids map) go to a FIFO SCAN list; everything else goes to a GEN list run as a CLOCK with a 2-bit frequency counter (kept in a hash map for GEN folios only; an access bumps the counter and does not move the folio). A non-scan access to a scan folio promotes it to GEN. Eviction drains the SCAN list head first (oldest, already consumed scan pages), then the GEN head.

Agent's title for the attempt: v3: GEN list LRU -> CLOCK with 2-bit freq counter (hash map only for GEN folios; accessed bumps freq, no list move). Scan FIFO unchanged. Tests whether frequency on GET side cuts bytes read without get_scan's throughput loss

Ancestor: get_scan.c, heavily rewritten (closest seed only 67% similar); lineage root is attempt #1 (get_scan.c). Lines that differ from the ancestor are highlighted.

```
// ============================================================================
// SECTION: BPF KERNEL CODE
// ============================================================================
// EVOLVE-BLOCK-START
// Scan-first FIFO + GET LRU (captain-nemo v2). Two lists, no per-folio map:
//  * SCAN list: folios added by a scan-thread pid (harness scan_pids map), FIFO.
//  * GEN list:  everything else, CLOCK with 2-bit freq (gen_freq map, GEN folios only).
// A non-scan access to a scan folio promotes it to GEN. Eviction drains the SCAN
// list head (oldest, already-consumed scan pages) first, then the GEN head.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, int);
    __type(value, bool);
    __uint(max_entries, 100);
} scan_pids SEC(".maps");

static inline bool is_scanning_pid(void)
{
    int pid = (int)(bpf_get_current_pid_tgid() & 0xFFFFFFFF);
    return bpf_map_lookup_elem(&scan_pids, &pid) != NULL;
}

static inline bool is_folio_relevant(struct folio *folio)
{
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;
    return inode_in_watchlist(folio->mapping->host->i_ino);
}

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, __u64);
    __type(value, u32);
    __uint(max_entries, 1 << 21);
} gen_freq SEC(".maps");

#define FREQ_MAX 3

__u64 scan_list;
__u64 gen_list;

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    scan_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (scan_list == 0)
        return -1;
    gen_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (gen_list == 0)
        return -1;
    return 0;
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio)
{
    if (!is_folio_relevant(folio))
        return;
    if (is_scanning_pid()) {
        bpf_cache_ext_list_add_tail(scan_list, folio);
    } else {
        __u64 key = (__u64)folio;
        u32 zero = 0;
        bpf_map_update_elem(&gen_freq, &key, &zero, BPF_ANY);
        bpf_cache_ext_list_add_tail(gen_list, folio);
    }
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio)
{
    if (!is_folio_relevant(folio))
        return;
    if (is_scanning_pid())
        return;
    __u64 key = (__u64)folio;
    u32 *freq = bpf_map_lookup_elem(&gen_freq, &key);
    if (freq) {
        if (*freq < FREQ_MAX)
            *freq += 1;
        return;
    }
    // Scan folio touched by a GET: promote to GEN with one credit.
    u32 one = 1;
    bpf_map_update_elem(&gen_freq, &key, &one, BPF_ANY);
    bpf_cache_ext_list_move(gen_list, folio, true);
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio)
{
    __u64 key = (__u64)folio;
    bpf_cache_ext_list_del(folio);
    bpf_map_delete_elem(&gen_freq, &key);
}

static int evict_iter_fn(int idx, struct cache_ext_list_node *node)
{
    struct folio *f = node->folio;
    if (!folio_test_uptodate(f) || !folio_test_lru(f))
        return CACHE_EXT_CONTINUE_ITER;
    if (folio_test_dirty(f) || folio_test_writeback(f) || folio_test_locked(f))
        return CACHE_EXT_CONTINUE_ITER;
    return CACHE_EXT_EVICT_NODE;
}

// CLOCK on GEN: a folio with credit loses one and goes to the tail; else evict.
static int gen_iter_fn(int idx, struct cache_ext_list_node *node)
{
    struct folio *f = node->folio;
    if (!folio_test_uptodate(f) || !folio_test_lru(f))
        return CACHE_EXT_CONTINUE_ITER;
    if (folio_test_dirty(f) || folio_test_writeback(f) || folio_test_locked(f))
        return CACHE_EXT_CONTINUE_ITER;
    __u64 key = (__u64)f;
    u32 *freq = bpf_map_lookup_elem(&gen_freq, &key);
    if (freq && *freq > 0) {
        *freq -= 1;
        return CACHE_EXT_CONTINUE_ITER;
    }
    return CACHE_EXT_EVICT_NODE;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };
    bpf_cache_ext_list_iterate_extended(memcg, scan_list, evict_iter_fn, &opts, eviction_ctx);
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    bpf_cache_ext_list_iterate_extended(memcg, gen_list, gen_iter_fn, &opts, eviction_ctx);
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
```

```
        return;
    bpf_cache_ext_list_iterate_extended(memcg, gen_list, gen_iter_fn, &opts, eviction_ctx);
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the GET-SCAN policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <signal.h>
#include <string.h>
#include <stdlib.h>
#include <stdint.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "evo_policy.skel.h"
#include "dir_watcher.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
    char *cgroup_path;
    uint64_t cgroup_size;   /* accepted for harness compatibility */
};

static struct argp_option options[] = { { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
                    { "cgroup_path", 'c', "PATH", 0,
                      "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)" },
                    { "cgroup_size", 's', "SIZE", 0,
      "Size of the cgroup in bytes (accepted for harness compatibility)" },
                    { 0 } };

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
    case 'c':
        args->cgroup_path = arg;
        break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno)
            args->cgroup_size = 0;
        break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static volatile sig_atomic_t exiting;

static void sig_handler(int signo)
{
    exiting = 1;
}

int main(int argc, char **argv)
{
    int ret = 1;
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    int cgroup_fd = -1;

libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

{
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }
}

// Parse command line arguments
struct cmdline_args args = { 0 };
struct argp argp = { options, parse_opt, 0, 0 };
argp_parse(&argp, argc, argv, 0, 0, &args);

// Validate arguments
if (args.watch_dir == NULL) {
    fprintf(stderr, "Missing required argument: watch_dir\n");
    return 1;
}

if (args.cgroup_path == NULL) {
    fprintf(stderr, "Missing required argument: cgroup_path\n");
    return 1;
}

// Does watch_dir exist?
if (access(args.watch_dir, F_OK) == -1) {
    fprintf(stderr, "Directory does not exist: %s\n",
        args.watch_dir);
    return 1;
}

// Get full path of watch_dir
char watch_dir_full_path[PATH_MAX];
if (realpath(args.watch_dir, watch_dir_full_path) == NULL) {
    perror("realpath");
    return 1;
}

// TODO: Enable longer length
if (strlen(watch_dir_full_path) > 128) {
    fprintf(stderr, "watch_dir path too long\n");
    return 1;
}

cgroup_fd = open(args.cgroup_path, O_RDONLY);
if (cgroup_fd < 0) {
    perror("Failed to open cgroup path");
    return 1;
}

// Open skel
skel = evo_policy_bpf__open();
if (skel == NULL) {
    perror("Failed to open BPF skeleton");
    goto cleanup;
}

// Set watch_dir
skel->rodata->watch_dir_path_len = strlen(watch_dir_full_path);
strcpy(skel->rodata->watch_dir_path, watch_dir_full_path);

// The benchmark harness creates and pins the scan_pids map before any policy loads
// (and removes the pin afterwards). Reuse it; do not pin our own.
{
    int scan_pids_fd = bpf_obj_get("/sys/fs/bpf/cache_ext/scan_pids");
    if (scan_pids_fd >= 0) {
        if (bpf_map__reuse_fd(skel->maps.scan_pids, scan_pids_fd))
            fprintf(stderr, "Failed to reuse scan_pids map fd\n");
        close(scan_pids_fd);
    } else {
        fprintf(stderr, "no scan_pids pin found\n");
    }
}

// Load programs
ret = evo_policy_bpf__load(skel);
if (ret) {
    perror("Failed to load BPF skeleton");
    goto cleanup;
}
```

```
    // Initialize inode_watchlist map
    ret = initialize_watch_dir_map(args.watch_dir,
                       bpf_map__fd(skel->maps.inode_watchlist), false);

    // Attach cache_ext_ops to the specific cgroup
    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        goto cleanup;
    }

    // Attach probes
    ret = evo_policy_bpf__attach(skel);
    if (ret) {
        perror("Failed to attach BPF programs");
        goto cleanup;
    }

    printf("evo_policy (GET-SCAN) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

one-seed / tanh (pc309): best policy #19 fefe1ef1 — score 0.755, 647.6 ops/s

Unmodified S3-FIFO except for one rule: a ghost-map hit (a folio re-referenced soon after eviction) re-enters the small probation queue with freq=1, so one more access promotes it, instead of going straight to the main queue. A 12-line diff.

Agent's title for the attempt: peak test (opposite of eval18): ghost hits go to small queue with freq=1 (one more access promotes) instead of straight to main

Ancestor: s3fifo.c, closest seed by line similarity (99%). Lines that differ from the ancestor are highlighted.

```
// ============================================================================
// SECTION: BPF KERNEL CODE
// ============================================================================
// EVOLVE-BLOCK-START
// S3-FIFO (ghost hits -> small, freq 1): a small FIFO (~10% of the cache) admits new folios; a folio re-accessed while in
// the small queue is promoted to the main queue, otherwise evicted. The main queue is
// CLOCK-like with a frequency counter capped at 3. A ghost map remembers recently evicted
// folios so a quick re-reference is admitted straight into main.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define ENOENT      2  /* include/uapi/asm-generic/errno-base.h */
#define INT64_MAX   (9223372036854775807LL)

// Set from userspace. In terms of number of pages.
// TODO: change

//#define CACHE_SIZE (((1ull << 30) * 2) / 4096)
#define CACHE_SIZE (((1ull << 20) * 200) / 4096)
const volatile size_t cache_size = 0;

struct folio_metadata {
    s64 freq;
    bool in_main;
};

struct ghost_entry {
    u64 address_space;
    u64 offset;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, u64);
    __type(value, struct folio_metadata);
    __uint(max_entries, 4000000);
} folio_metadata_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ghost_entry);
    __type(value, u8);
    //__uint(max_entries, CACHE_SIZE); // TODO: change
    __uint(map_flags, BPF_F_NO_COMMON_LRU);  // Per-CPU LRU eviction logic
} ghost_map SEC(".maps");

static u64 main_list;
static u64 small_list;

/*
 * This is an approximate value based on what we choose to evict, not what is
 * actually evicted.
 */
static s64 small_list_size = 0;
static s64 main_list_size = 0;

static inline bool is_folio_relevant(struct folio *folio) {
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;
    return inode_in_watchlist(folio->mapping->host->i_ino);
}

static inline struct folio_metadata *get_folio_metadata(struct folio *folio) {
    u64 key = (u64)folio;
    return bpf_map_lookup_elem(&folio_metadata_map, &key);
}

/*
 * Check if a folio is in the ghost map and delete the ghost entry.
 * We only check if an element is in the ghost map on inserting into the cache.
 * Relies on bpf_map_delete_elem() returning -ENOENT if the element is not found.
 */
static inline bool folio_in_ghost(struct folio *folio) {
    struct ghost_entry key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    // TODO: handle non-ENOENT errors
    return bpf_map_delete_elem(&ghost_map, &key) != -ENOENT;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    main_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (main_list == 0) {
        bpf_printk("cache_ext: init: Failed to create main_list\n");
        return -1;
    }
    bpf_printk("cache_ext: Created main_list: %llu\n", main_list);

    small_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (small_list == 0) {
        bpf_printk("cache_ext: init: Failed to create small_list\n");
        return -1;
    }
    bpf_printk("cache_ext: Created small_list: %llu\n", small_list);

    return 0;
}

static s64 bpf_s3fifo_score_main_fn(struct cache_ext_list_node *a) {
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return INT64_MAX;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return INT64_MAX;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return INT64_MAX;
    }
    s64 freq = __sync_sub_and_fetch(&data->freq, 1);
    if (freq < 0)
        data->freq = 0;

    return freq;
}

static int bpf_s3fifo_score_small_fn(int idx, struct cache_ext_list_node *a)
{
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Move to main list if freq > 1
    if (data->freq > 1) {
        data->in_main = true;
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Else, evict
    return CACHE_EXT_EVICT_NODE;
}

static void evict_main(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
```

```
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct sampling_options opts = {
        .sample_size = 10,
    };

    if (bpf_cache_ext_list_sample(memcg, main_list, bpf_s3fifo_score_main_fn, &opts,
                      eviction_ctx)) {
        bpf_printk("cache_ext: evict: Failed to sample main_list\n");
        return;
    }

    // if (__sync_sub_and_fetch(&main_list_size, eviction_ctx->nr_folios_to_evict) < 0)
    //  main_list_size = 0;
}

#define MAIN_ITER_FN(id)                                \
static int bpf_s3fifo_score_main_iter_fn_##id(int idx, struct cache_ext_list_node *a)   \
{                                                       \
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))        \
        return CACHE_EXT_CONTINUE_ITER;                 \
                                                        \
    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))       \
        return CACHE_EXT_CONTINUE_ITER;                 \
                                                        \
    struct folio_metadata *data = get_folio_metadata(a->folio);             \
    if (!data) {                                        \
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");        \
        return CACHE_EXT_CONTINUE_ITER;                 \
    }                                                   \
                                                        \
    s64 freq = __sync_sub_and_fetch(&data->freq, 1);                \
    if (freq < id) {                                    \
        /*data->freq = 0;*/                             \
        return CACHE_EXT_EVICT_NODE;                    \
    }                                                   \
                                                        \
    return CACHE_EXT_CONTINUE_ITER;                     \
}

MAIN_ITER_FN(0)
MAIN_ITER_FN(1)
MAIN_ITER_FN(2)
MAIN_ITER_FN(3)

static void evict_main_iter(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_0, &opts,
                        eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_1, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_2, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_3, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    }
}

static void evict_small(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 1, move to main list, otherwise evict.
     * (When evicting, move to tail in the meantime).
     *
     * Use the iterate interface.
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = main_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, small_list, bpf_s3fifo_score_small_fn, &opts,
                            eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate small_list\n");
        return;
    }

    if (__sync_fetch_and_sub(&small_list_size, opts.nr_folios_continue) < 0)
        small_list_size = 0;

    if (__sync_fetch_and_add(&main_list_size, opts.nr_folios_continue) < 0)
        main_list_size = opts.nr_folios_continue;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    // bpf_printk("cache_ext: evict_folios: main_list_size: %lld, small_list_size: %lld, cache_size: %lld\n",
    //     main_list_size, small_list_size, cache_size);
    if (small_list_size >= cache_size / 15 || main_list_size <= 2 * small_list_size)
        evict_small(eviction_ctx, memcg);
    else
        evict_main_iter(eviction_ctx, memcg);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    struct folio_metadata *data = get_folio_metadata(folio);
    if (!data) {
        bpf_printk("cache_ext: accessed: Failed to get metadata\n");
        return;
    }

    // Cap frequency at 3
    if (__sync_add_and_fetch(&data->freq, 1) > 3)
        data->freq = 3;
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
    u64 key = (u64)folio;
    u8 ghost_val = 0;

    // if (bpf_cache_ext_list_del(folio)) {
    //  bpf_printk("cache_ext: Failed to delete folio from sampling_list\n");
    //  return;
    // }

    struct ghost_entry ghost_key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };

    // Don't return early, we want to delete the folio metadata regardless
    if (bpf_map_update_elem(&ghost_map, &ghost_key, &ghost_val, BPF_ANY))
        bpf_printk("cache_ext: evicted: Failed to add to ghost_map\n");

    struct folio_metadata *data = get_folio_metadata(folio);
```

```
    if (!data) {
        //bpf_printk("cache_ext: evicted: Failed to get metadata\n");
        return;
    }

    if (data->in_main)
        __sync_fetch_and_sub(&main_list_size, 1);
    else
        __sync_fetch_and_sub(&small_list_size, 1);

    bpf_map_delete_elem(&folio_metadata_map, &key);

    // if (bpf_map_delete_elem(&folio_metadata_map, &key))
    //  bpf_printk("cache_ext: evicted: Failed to delete metadata\n");
}

/*
 * If folio is in the ghost map, add to tail of main list, otherwise add to tail
 * of small list.
 */
void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    u64 key = (u64)folio;
    struct folio_metadata new_meta = {
        .freq = 0,
    };

    u64 list_to_add;
    if (folio_in_ghost(folio)) {
        // Peak test: ghost hits re-enter probation, but one access promotes them.
        list_to_add = small_list;
        new_meta.in_main = false;
        new_meta.freq = 1;
        __sync_fetch_and_add(&small_list_size, 1);
    } else {
        list_to_add = small_list;
        new_meta.in_main = false;
        __sync_fetch_and_add(&small_list_size, 1);
    }

    if (bpf_cache_ext_list_add_tail(list_to_add, folio)) {
        // TODO: add back to ghost_map?
        bpf_printk("cache_ext: added: Failed to add folio to main_list\n");
        return;
    }

    if (bpf_map_update_elem(&folio_metadata_map, &key, &new_meta, BPF_ANY)) {
        // TODO: add back to ghost_map? + error check delete call?
        bpf_cache_ext_list_del(folio);
        bpf_printk("cache_ext: added: Failed to create folio metadata\n");
        return;
    }
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the S3-FIFO policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <stdlib.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "dir_watcher.h"
#include "evo_policy.skel.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_size <size> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
        uint64_t cgroup_size;
        char *cgroup_path;
};

static struct argp_option options[] = {
    { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
        {"cgroup_size", 's', "SIZE", 0, "Size of the cgroup"},
        {"cgroup_path", 'c', "PATH", 0, "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)"},
    { 0 },
};

static const uint64_t page_size = 4096;

static volatile sig_atomic_t exiting;

static void sig_handler(int signo) {
    exiting = 1;
}

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
        case 's':
                // TODO: move this to parse_args()
                errno = 0;
                args->cgroup_size = strtoull(arg, NULL, 10);
                if (errno)
                        args->cgroup_size = 0;

                break;
        case 'c':
                args->cgroup_path = arg;
                break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static int parse_args(int argc, char **argv, struct cmdline_args *args) {
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, args);

    if (args->watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args->cgroup_size == 0) {
            fprintf(stderr, "Invalid cgroup size\n");
            return 1;
    }

    if (args->cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    return 0;
}

/*
 * Validate watch_dir
 *
 * watch_dir_full_path must be able to hold PATH_MAX bytes.
 */
static int validate_watch_dir(const char *watch_dir, char *watch_dir_full_path) {
    // Does watch_dir exist?
    if (access(watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n", watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    if (realpath(watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
```

```
        return 1;
    }

    // BPF policy restriction
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    struct sigaction sa;
    char watch_dir_path[PATH_MAX];
    int cgroup_fd = -1;
    int ret = 1;

    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    if (parse_args(argc, argv, &args))
        return 1;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;

    // Install signal handler
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }

    if (validate_watch_dir(args.watch_dir, watch_dir_path))
        return 1;

    // Open cgroup directory early
    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    skel = evo_policy_bpf__open();
    if (!skel) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    // Set cache size in terms of number of pages. Assumes uniform page size.
    skel->rodata->cache_size = args.cgroup_size / page_size;
    fprintf(stderr, "Cgroup size: %lu bytes\n", args.cgroup_size);
    fprintf(stderr, "Cache size: %lu pages\n", skel->rodata->cache_size);

    // Resize ghost_map
    if (bpf_map__set_max_entries(skel->maps.ghost_map, skel->rodata->cache_size)) {
        perror("Failed to resize ghost_map");
        ret = 1;
        goto cleanup;
    }

    // Set watch_dir
    watch_dir_path_len_map(skel) = strlen(watch_dir_path);
    strcpy(watch_dir_path_map(skel), watch_dir_path);

    if (evo_policy_bpf__load(skel)) {
        perror("Failed to load BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    if (initialize_watch_dir_map(watch_dir_path, bpf_map__fd(inode_watchlist_map(skel)), true)) {
        perror("Failed to initialize watch_dir map");
        ret = 1;
        goto cleanup;
    }

    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        ret = 1;
        goto cleanup;
    }

    // This is necessary for the dir_watcher functionality
    if (evo_policy_bpf__attach(skel)) {
        perror("Failed to attach BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    printf("evo_policy (S3-FIFO) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

all-seeds / max-throughput (pc319): best policy #18 e8ecbcfc — score 679.3, 679.3 ops/s

MGLRU (multi-generational LRU port) with one change: folios added by a scan thread (found through the pinned scan_pids map, reused by the loader) are inserted into the youngest generation instead of the normal placement. The agent reported that every variant that demoted scan folios lost throughput, so it tried protecting them.

Agent's title for the attempt: contrarian: MGLRU + scan-thread folios inserted at youngest gen (max_seq), refs counted normally. Every scan-demoting variant lost (mglru 658->634 w/ scan@oldest; s3fifo 637->619-632)

Ancestor: mglru.c, closest seed by line similarity (98%). Lines that differ from the ancestor are highlighted.

```
// ===================================================================================
// SECTION: BPF KERNEL CODE
// ===================================================================================
// EVOLVE-BLOCK-START
// MGLRU: a BPF port of Linux's multi-generational LRU. Folios sit in generations (by age)
// and tiers (by access frequency); aging advances the generation counters and eviction
// takes from the oldest generation, with a ghost map feeding refault information back.
// captain-nemo: folios added by the scan thread (scan_pids) go to the YOUNGEST generation:
// demoting scan pages consistently cost throughput (evals 10/11/16/17), so test protecting them.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define ARRAY_SIZE(x) (sizeof(x) / sizeof((x)[0]))
#define INT64_MAX (9223372036854775807LL)

// #define DEBUG
#ifdef DEBUG
#define dbg_printk(fmt, ...) bpf_printk(fmt, ##__VA_ARGS__)
#else
#define dbg_printk(fmt, ...)
#endif

#define ENOENT      2  /* include/uapi/asm-generic/errno-base.h */
#define atomic_long_read(ptr) __sync_fetch_and_add(ptr, 0)
#define atomic_long_zero(ptr) __sync_fetch_and_and(ptr, 0)
#define atomic_long_store(ptr, val) __sync_lock_test_and_set(ptr, val)

#define DEFINE_MIN_SEQ(lrugen) \
    unsigned long min_seq = READ_ONCE(lrugen->min_seq)
#define DEFINE_MAX_SEQ(lrugen) \
    unsigned long max_seq = READ_ONCE(lrugen->max_seq)

//////////
// Maps //
//////////

#define MAX_NR_FOLIOS 4000000
#define MAX_NR_GHOST_ENTRIES 400000

struct folio_metadata {
    s64 accesses;
    s64 gen;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, __u64);
    __type(value, struct folio_metadata);
    __uint(max_entries, MAX_NR_FOLIOS);
} folio_metadata_map SEC(".maps");

//////////////////
// Ghost Enties //
//////////////////

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ghost_entry);
    __type(value, u8);
    __uint(max_entries, MAX_NR_GHOST_ENTRIES);
    __uint(map_flags, BPF_F_NO_COMMON_LRU);  // Per-CPU LRU eviction logic
} ghost_map SEC(".maps");

struct ghost_entry {
    u64 address_space;
    u64 offset;
};

static inline void insert_ghost_entry_for_folio(struct folio *folio, int tier) {
    struct ghost_entry ghost_key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    u8 ghost_val = (u8) tier;
    if (bpf_map_update_elem(&ghost_map, &ghost_key, &ghost_val, BPF_ANY))
            bpf_printk("cache_ext: evicted: Failed to add to ghost_map\n");
}

/*
 * Check if a folio is in the ghost map and delete the ghost entry.
 * We only check if an element is in the ghost map on inserting into the cache.
 * Relies on bpf_map_delete_elem() returning -ENOENT if the element is not found.
 */
static inline int folio_in_ghost(struct folio *folio) {
    struct ghost_entry key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    u8 *tier;
    tier = bpf_map_lookup_elem(&ghost_map, &key);
    if (tier == NULL) {
        return -1;
    }
    // TODO: handle non-ENOENT errors
    bpf_map_delete_elem(&ghost_map, &key);
    return (int)(*tier);
}

////////////////////////////////////////////////////////////////////////////////////
//  _  _____ _____ ____  _   _ _____ _     ____ ___ _____ ____  _____ ____  ____  //
// | |/ / ____|  _ \| \ | | ____| |     |  _ \_ _|  \/  | _ \ / _ \|  _ \|  _ \  //
// | ' /|  _| | |_) |  \| |  _| | |     | |_) | || |\/| | |_) | | | | |_) | |_) | //
// | . \| |___|  _ <| |\  | |___| |___  |  __/| || |  | |  __/| |_| |  _ <|  __/  //
// |_|\_\_____|_| \_\_| \_|_____|_____| |___|_|  |_|  |_|_|    \___/|_| \_\_|     //
////////////////////////////////////////////////////////////////////////////////////

// Constants

#define MAX_NR_TIERS 4
#define MIN_NR_GENS 2
#define MAX_NR_GENS 4
#define NR_HIST_GENS 1
#define MIN_LRU_BATCH 64
// Global policy metadata
struct mglru_global_metadata {
    struct bpf_spin_lock lock;
    unsigned long max_seq;
    unsigned long min_seq;
    s64 evicted[MAX_NR_TIERS];
    s64 refaulted[MAX_NR_TIERS];
    s64 tier_selected[MAX_NR_TIERS];
    s64 success_evicted;
    s64 failed_evicted;
    unsigned long avg_refaulted[MAX_NR_TIERS];
    unsigned long avg_total[MAX_NR_TIERS];
    unsigned long protected[MAX_NR_TIERS - 1];
    long nr_pages[MAX_NR_GENS];
};

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __type(key, int);
    __type(value, struct mglru_global_metadata);
    __uint(max_entries, 1);
} mglru_global_metadata_map SEC(".maps");

#define DEFINE_LRUGEN_void                                                  \
    struct mglru_global_metadata *lrugen;                                   \
    int key__ = 0;                                                          \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);       \
```

```
    if (!lrugen) {                                                  \
        bpf_printk(                                                 \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return;                                                     \
    }

#define DEFINE_LRUGEN_int                                               \
    struct mglru_global_metadata *lrugen;                               \
    int key__ = 0;                                                      \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);   \
    if (!lrugen) {                                                      \
        bpf_printk(                                                 \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return -1;                                                  \
    }

#define DEFINE_LRUGEN_bool                                              \
    struct mglru_global_metadata *lrugen;                               \
    int key__ = 0;                                                      \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);   \
    if (!lrugen) {                                                      \
        bpf_printk(                                                 \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return false;                                               \
    }

// TODO: Hook these up with the eventual policy
// Helpers to update metadata

#define assert_valid_tier_0(tier_idx)                               \
    if (tier_idx < 0 || tier_idx >= MAX_NR_TIERS) {             \
        bpf_printk("cache_ext: Invalid tier index %d\n", \
                tier_idx);                                  \
        return;                                             \
    }

#define assert_valid_gen_0(gen_idx)                                     \
    if (gen_idx < 0 || gen_idx >= MAX_NR_GENS) {                        \
        bpf_printk("cache_ext: Invalid gen index %d\n", gen_idx); \
        return;                                                     \
    }

#define assert_valid_tier_1(tier_idx)                               \
    if (tier_idx < 0 || tier_idx >= MAX_NR_TIERS) {             \
        bpf_printk("cache_ext: Invalid tier index %d\n", \
                tier_idx);                                  \
        return -1;                                          \
    }

#define assert_valid_gen_1(gen_idx)                                     \
    if (gen_idx < 0 || gen_idx >= MAX_NR_GENS) {                        \
        bpf_printk("cache_ext: Invalid gen index %d\n", gen_idx); \
        return -1;                                                  \
    }

inline void update_refaulted_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                  s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->refaulted[tier_idx], delta);
}

inline void update_evicted_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->evicted[tier_idx], delta);
}

inline void update_nr_pages_stat(struct mglru_global_metadata *lrugen, unsigned int gen_idx,
                 s64 delta)
{
    assert_valid_gen_0(gen_idx);
    __sync_fetch_and_add(&lrugen->nr_pages[gen_idx], delta);
}

inline void update_tier_selected_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                 s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->tier_selected[tier_idx], delta);
}


inline s64 read_refaulted_stat(struct mglru_global_metadata *lrugen, int tier_idx)
{
    assert_valid_tier_1(tier_idx);
    return max(0, atomic_long_read(&lrugen->refaulted[tier_idx]));
}

inline s64 read_evicted_stat(struct mglru_global_metadata *lrugen, int tier_idx)
{
    assert_valid_tier_1(tier_idx);
    return max(0, atomic_long_read(&lrugen->evicted[tier_idx]));
}

inline s64 read_nr_pages_stat(struct mglru_global_metadata *lrugen, unsigned int gen_idx)
{
    assert_valid_gen_1(gen_idx);
    return max(0, atomic_long_read(&lrugen->nr_pages[gen_idx]));
}

// Invoke when promoting a folio in the eviction iteration
// See: https://github.com/cache-ext/linux-cachestream/blob/c22ffcac6b53ef4054483070fb902895ef10fd12/mm/vmscan.c#L4941-L4952
inline void update_protected_stat(struct mglru_global_metadata *lrugen, int tier_idx,
                  s64 delta)
{
    __sync_fetch_and_add(&lrugen->protected[tier_idx - 1], delta);
}

// Gen lists
static __u64 mglru_lists[MAX_NR_GENS];

/******************************************************************************
 *                          PID controller
 ******************************************************************************/

/*
 * A feedback loop based on Proportional-Integral-Derivative (PID) controller.
 *
 * The P term is refaulted/(evicted+protected) from a tier in the generation
 * currently being evicted; the I term is the exponential moving average of the
 * P term over the generations previously evicted, using the smoothing factor
 * 1/2; the D term isn't supported.
 *
 * The setpoint (SP) is always the first tier of one type; the process variable
 * (PV) is either any tier of the other type or any other tier of the same
 * type.
 *
 * The error is the difference between the SP and the PV; the correction is to
 * turn off protection when SP>PV or turn on protection when SP<PV.
 *
 * For future optimizations:
 * 1. The D term may discount the other two terms over time so that long-lived
 *    generations can resist stale information.
 */

 // Avoid name conflict with kernel struct ctrl_pos if MGLRU is enabled
 struct ctrl_pos___x {
    unsigned long refaulted;
    unsigned long total;
    int gain;
};

static inline void read_ctrl_pos(struct mglru_global_metadata *lrugen, int tier,
                 int gain, struct ctrl_pos___x *pos)
{
    pos->refaulted = lrugen->avg_refaulted[tier] +
             __sync_fetch_and_add(&lrugen->refaulted[tier], 0);
    pos->total = lrugen->avg_total[tier] +
             __sync_fetch_and_add(&lrugen->evicted[tier], 0);
    if (tier)
        pos->total += lrugen->protected[tier - 1];
    pos->gain = gain;
}

// Needs the LRU lock. Why???
static inline void reset_ctrl_pos(struct mglru_global_metadata *lrugen, bool carryover)
{
    int tier;
    bool clear = carryover ? NR_HIST_GENS == 1 : NR_HIST_GENS > 1;

    // lockdep_assert_held(&lruvec->lru_lock);

    if (!carryover && !clear)
        return;

    for (tier = 0; tier < MAX_NR_TIERS; tier++) {
        if (carryover) {
            unsigned long sum;

            sum = lrugen->avg_refaulted[tier] +
```

```
                atomic_long_read(&lrugen->refaulted[tier]);
            WRITE_ONCE(lrugen->avg_refaulted[tier], sum / 2);

            sum = lrugen->avg_total[tier] +
                atomic_long_read(&lrugen->evicted[tier]);
            if (tier)
                sum += lrugen->protected[tier - 1];
            WRITE_ONCE(lrugen->avg_total[tier], sum / 2);
        }

        if (clear) {
            atomic_long_zero(&lrugen->refaulted[tier]);
            atomic_long_zero(&lrugen->evicted[tier]);
            if (tier)
                WRITE_ONCE(lrugen->protected[tier - 1], 0);
        }
    }
}

static inline bool positive_ctrl_err(struct ctrl_pos___x *sp, struct ctrl_pos___x *pv)
{
    /*
     * Return true if the PV has a limited number of refaults or a lower
     * refaulted/total than the SP.
     */
    return pv->refaulted < MIN_LRU_BATCH ||
           pv->refaulted * (sp->total + MIN_LRU_BATCH) * sp->gain <=
               (sp->refaulted + 1) * pv->total * pv->gain;
}

/////////////////////////////////////
// Utils that use the PID controller //
/////////////////////////////////////

static inline int get_tier_idx(struct mglru_global_metadata *lrugen)
{
    int tier;
    struct ctrl_pos___x sp, pv;

    /*
     * To leave a margin for fluctuations, use a larger gain factor (1:2).
     * This value is chosen because any other tier would have at least twice
     * as many refaults as the firsfirst tier.
     */
    read_ctrl_pos(lrugen, 0, 1, &sp);
    for (tier = 1; tier < MAX_NR_TIERS; tier++) {
        read_ctrl_pos(lrugen, tier, 2, &pv);
        if (!positive_ctrl_err(&sp, &pv))
            break;
    }

    return tier - 1;
}

// static inline int get_tier_idx(struct mglru_global_metadata *lrugen)
// {
//  // TODO: Swap this with the real implementation once we add refault tracking
//  // support.
//  return 0;
// }

static inline void folio_inc_refs(struct folio *folio)
{
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata) {
        bpf_printk(
            "cache_ext: Tried to inc refs but folio not found in map.\n");
        return;
    }
    __sync_fetch_and_add(&metadata->accesses, 1);
}

static inline int folio_lru_refs(struct folio *folio)
{
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata)
        return -1;

    return atomic_long_read(&metadata->accesses);
}
```

```
static inline unsigned int lru_gen_from_seq(unsigned long seq)
{
    return seq % MAX_NR_GENS;
}

static inline bool folio_test_active(struct folio *folio)
{
    return folio_lru_refs(folio) >= 2;
}

// Only counts up to 3.
static inline int order_base_2(int n)
{
    if (n <= 1) {
        return 0;
    } else if (n <= 3) {
        return 1;
    } else if (n <= 7) {
        return 2;
    } else if (n <= 8) {
        return 3;
    }
    return 3;
}

static inline int lru_tier_from_refs(int refs)
{
    /* see the comment in folio_lru_refs() */
    // kernel does some off-by-one calculation here, we remove the +1
    // return order_base_2(refs + 1);
    return order_base_2(refs);
}

static inline bool gen_within_limits(unsigned int gen)
{
    return 0 <= gen && gen <= MAX_NR_GENS;
}

static inline int get_nr_gens(struct mglru_global_metadata *lrugen)
{
    return lrugen->max_seq - lrugen->min_seq + 1;
}

static inline bool gen_almost_empty(struct mglru_global_metadata *lrugen,
                    int min_seq)
{
    int oldest_gen = lru_gen_from_seq(min_seq);
    int nr_folios = read_nr_pages_stat(lrugen, oldest_gen);
    int threshold = 4;
    return nr_folios <= threshold;
}

// TODO: This is supposed to run with a lock.
static inline bool lru_gen_add_folio(struct folio *folio, bool scan)
{
    unsigned long seq;
    if (folio_test_unevictable(folio)) {
        bpf_printk("cache_ext: Unevictable folio\n");
        return false;
    }

    DEFINE_LRUGEN_bool;
    DEFINE_MIN_SEQ(lrugen);
    DEFINE_MAX_SEQ(lrugen);
    /*
     * There are four common cases for this page:
     * 1. If it's hot, i.e., freshly faulted in, add it to the youngest
     *    generation, and it's protected over the rest below.
     * 2. If it can't be evicted immediately, i.e., a dirty page pending
     *    writeback, add it to the second youngest generation.
     * 3. If it should be evicted first, e.g., cold and clean from
     *    folio_rotate_reclaimable(), add it to the oldest generation.
     * 4. Everything else falls between 2 & 3 above and is added to the
     *    second oldest generation if it's considered inactive, or the
     *    oldest generation otherwise. See lru_gen_is_active().
     */
    if (scan)
        seq = max_seq;
    else if (folio_test_active(folio))
        seq = max_seq;
    else if ((folio_test_reclaim(folio) &&
          (folio_test_dirty(folio) || folio_test_writeback(folio))))
        seq = max_seq - 1;
    else if (min_seq + MIN_NR_GENS >= max_seq)
        seq = min_seq;
```

```
    else
        seq = min_seq + 1;

    unsigned int gen = lru_gen_from_seq(seq);
    if (!gen_within_limits(gen)) {
        bpf_printk(
            "cache_ext: Invalid gen %d (min_seq=%lu, max_seq=%lu)\n",
            gen, min_seq, max_seq);
        return false;
    }

    // Update policy metadata
    struct folio_metadata metadata = { .accesses = 1, .gen = gen };
    __u64 key = (__u64)folio;
    int ret = bpf_map_update_elem(&folio_metadata_map, &key, &metadata,
                      BPF_ANY);
    if (ret != 0) {
        bpf_printk("cache_ext: Failed to save folio metadata\n");
        return false;
    }
    update_nr_pages_stat(lrugen, gen, folio_nr_pages(folio));

    // Update refaulted stats
    ret = folio_in_ghost(folio);
    if (ret >= 0) {
        int tier = ret;
        update_refaulted_stat(lrugen, tier, 1);
    }

    // lru_gen_update_size(lruvec, folio, -1, gen);
    ret = bpf_cache_ext_list_add(mglru_lists[gen], folio);
    if (ret != 0) {
        bpf_printk(
            "cache_ext: Failed to add folio to mglru_lists[%d]\n",
            gen);
        return false;
    }

    return true;
}

static inline bool should_run_aging(struct mglru_global_metadata *lrugen,
                    unsigned long max_seq)
{
    unsigned int gen;
    unsigned long old = 0;
    unsigned long young = 0;
    unsigned long total = 0;
    DEFINE_MIN_SEQ(lrugen);

    /* whether this lruvec is completely out of cold folios */
    if (min_seq + MIN_NR_GENS > max_seq) {
        return true;
    }

    unsigned long seq;
    int max_iter =  min(MAX_NR_GENS, max_seq - min_seq);
    for (int i = 0; i < max_iter; i++) {
        seq = min_seq + i;
        unsigned long size = 0;

        gen = lru_gen_from_seq(seq);

        size += max(read_nr_pages_stat(lrugen, gen), 0L);

        total += size;
        if (seq == max_seq)
            young += size;
        else if (seq + MIN_NR_GENS == max_seq)
            old += size;
    }

    /*
     * The aging tries to be lazy to reduce the overhead, while the eviction
     * stalls when the number of generations reaches MIN_NR_GENS. Hence, the
     * ideal number of generations is MIN_NR_GENS+1.
     */
    if (min_seq + MIN_NR_GENS < max_seq)
        return false;

    /*
     * It's also ideal to spread pages out evenly, i.e., 1/(MIN_NR_GENS+1)
     * of the total number of pages for each generation. A reasonable range
     * for this average portion is [1/MIN_NR_GENS, 1/(MIN_NR_GENS+2)]. The
     * aging cares about the upper bound of hot pages, while the eviction
     * cares about the lower bound of cold pages.
     */
    if (young * MIN_NR_GENS > total)
        return true;
    if (old * (MIN_NR_GENS + 2) < total)
        return true;

    return false;
}

static inline bool try_to_inc_min_seq(struct mglru_global_metadata *lrugen)
{
    // TODO: Check if the min_seq list is empty and if it is, increase min_seq.
    DEFINE_MIN_SEQ(lrugen);
    if (!gen_almost_empty(lrugen, min_seq)) {
        return false;
    }
    WRITE_ONCE(lrugen->min_seq, lrugen->min_seq + 1);
    reset_ctrl_pos(lrugen, true);
    return true;
}

static inline bool try_to_inc_max_seq(struct mglru_global_metadata *lrugen)
{
    // int prev, next;
    // spin_lock_irq(&lruvec->lru_lock);

    if (get_nr_gens(lrugen) == MAX_NR_GENS) {
        // Try to increase min_seq
        int ret = try_to_inc_min_seq(lrugen);
        if (!ret) {
            return false;
        }
    }
    // spin_unlock_irq(&lruvec->lru_lock);

    // TODO: Do we need this?
    /*
     * Update the active/inactive LRU sizes for compatibility. Both sides of
     * the current max_seq need to be covered, since max_seq+1 can overlap
     * with min_seq[LRU_GEN_ANON] if swapping is constrained. And if they do
     * overlap, cold/hot inversion happens.
     */
    // prev = lru_gen_from_seq(lrugen->max_seq - 1);
    // next = lru_gen_from_seq(lrugen->max_seq + 1);

    // for (type = 0; type < ANON_AND_FILE; type++) {
    //  for (zone = 0; zone < MAX_NR_ZONES; zone++) {
    //      enum lru_list lru = type * LRU_INACTIVE_FILE;
    //      long delta = lrugen->nr_pages[prev] -
    //               lrugen->nr_pages[next];

    //      if (!delta)
    //          continue;

    //      __update_lru_size(lruvec, lru, zone, delta);
    //      __update_lru_size(lruvec, lru + LRU_ACTIVE, zone, -delta);
    //  }
    // }

    reset_ctrl_pos(lrugen, false);

    // We don't use the timestamp metadata for our MGLRU
    /* make sure preceding modifications appear */
    WRITE_ONCE(lrugen->max_seq, lrugen->max_seq + 1);
    return true;

    // spin_unlock_irq(&lruvec->lru_lock);
}

////////////////////////////////////////////////////////
//  ____   _     ____ _   _ _____     _______ _____  //
// |  _ \ / \   / ___| | | | ____|   | ____\ \/ /_ _| //
// | |_) / _ \ | |   | |_| |  _|     |  _|  \  / | |  //
// |  __/ ___ \| |___|  _  | |___    | |___ /  \ | |  //
// |_| /_/   \_\\____|_| |_|_____|___|_____/_/\_\___| //
//                              |_____|               //
////////////////////////////////////////////////////////


////////////
// Policy //
////////////


struct eviction_metadata {
    __u64 curr_gen;
```

```
    __u64 next_gen;
    __u64 iter_reached;
    __u64 tier_threshold;
};
struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 1);
    __type(key, u32);
    __type(value, struct eviction_metadata);
} mglru_percpu_array SEC(".maps");


struct eviction_metadata * get_eviction_metadata() {
    u32 key = 0;
    return bpf_map_lookup_elem(&mglru_percpu_array, &key);
}

void set_eviction_metadata(struct eviction_metadata *eviction_meta) {
    u32 key = 0;
    int ret = bpf_map_update_elem(&mglru_percpu_array, &key, eviction_meta, BPF_ANY);
    if (ret < 0) {
        bpf_printk("cache_ext: Failed to update eviction metadata\n");
    }
}

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, int);
    __type(value, bool);
    __uint(max_entries, 100);
} scan_pids SEC(".maps");

static inline bool is_scanning_pid(void)
{
    int pid = (int)(bpf_get_current_pid_tgid() & 0xFFFFFFFF);
    return bpf_map_lookup_elem(&scan_pids, &pid) != NULL;
}

inline bool is_folio_relevant(struct folio *folio)
{
    if (!folio) {
        return false;
    }
    if (folio->mapping == NULL) {
        return false;
    }
    if (folio->mapping->host == NULL) {
        return false;
    }
    bool res = inode_in_watchlist(folio->mapping->host->i_ino);
    return res;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    DEFINE_LRUGEN_int;
    WRITE_ONCE(lrugen->max_seq, MIN_NR_GENS + 1);
    for (int i = 0; i < MAX_NR_GENS; i++) {
        __u64 list_ptr = bpf_cache_ext_ds_registry_new_list(memcg);
        if (list_ptr == 0) {
                    bpf_printk(
            "cache_ext: Failed to allocate list for gen %d\n",
            i);
            return -1;
        }
        mglru_lists[i] = list_ptr;
    }
    return 0;
}


// MGLRU iteration function. Logic is mainly ported from sort_folio.
static int mglru_iter_fn(int idx, struct cache_ext_list_node *a)
{
    // We skip the following cases from sort_folio since we asssume we don't
    // accept these folios in this policy:
    // - Unevictable folios
    // - Swap-backed folios
    // - Promoted folios (these only appear through PTE accesses,
    //                    fd-accessed folios are promoted based on their tier)

    struct mglru_global_metadata *lrugen;
    int key__ = 0;
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);
    if (!lrugen) {
        bpf_printk(
            "cache_ext: Failed to lookup lrugen metadata\n");
        return CACHE_EXT_EVICT_NODE;
    }

    struct eviction_metadata *eviction_meta = get_eviction_metadata();
    if (!eviction_meta) {
        bpf_printk("cache_ext: iter_fn: Failed to get eviction metadata\n");
        return CACHE_EXT_EVICT_NODE;
    }
    eviction_meta->iter_reached = idx;

    // Get folio metadata
    __u64 key = (__u64)a->folio;
    struct folio_metadata *meta =
        bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!meta) {
        bpf_printk("cache_ext: iter_fn: Failed to get metadata\n");
        // TODO: Maybe we should evict it instead?
        return CACHE_EXT_EVICT_NODE;
    }

    int tier_threshold = eviction_meta->tier_threshold;
    if (tier_threshold > MAX_NR_TIERS || tier_threshold < 0) {
        bpf_printk("cache_ext: Invalid tier threshold %d\n", tier_threshold);
    }
    // int tier_threshold = 2;
    int tier = lru_tier_from_refs(atomic_long_read(&meta->accesses));

    /* protected */
    if (tier > tier_threshold) {
        update_protected_stat(lrugen, tier, folio_nr_pages(a->folio));
        // promote to next gen
        // TODO: Update nr_pages stats
        int num_pages = folio_nr_pages(a->folio);
        update_nr_pages_stat(lrugen, eviction_meta->curr_gen, -num_pages);
        update_nr_pages_stat(lrugen, eviction_meta->next_gen, num_pages);
        atomic_long_store(&meta->gen, eviction_meta->next_gen);
        return CACHE_EXT_CONTINUE_ITER;
    }

    /* waiting for writeback */
    if (folio_test_locked(a->folio) || folio_test_writeback(a->folio) ||
        folio_test_dirty(a->folio)) {
        // promote to next gen
        int num_pages = folio_nr_pages(a->folio);
        update_nr_pages_stat(lrugen, eviction_meta->curr_gen, -num_pages);
        update_nr_pages_stat(lrugen, eviction_meta->next_gen, num_pages);
        atomic_long_store(&meta->gen, eviction_meta->next_gen);
        return CACHE_EXT_CONTINUE_ITER;
    }
    return CACHE_EXT_EVICT_NODE;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    DEFINE_LRUGEN_void;

    bool inc_max_seq_failed = false;
    bpf_spin_lock(&lrugen->lock);
    DEFINE_MIN_SEQ(lrugen);
    DEFINE_MAX_SEQ(lrugen);
    if (should_run_aging(lrugen, max_seq)) {
        if (!try_to_inc_max_seq(lrugen)){
            // Workaround: Not allowed to call bpf_printk under spinlock
            inc_max_seq_failed = true;
        }
    }
    if (max_seq - min_seq > MIN_NR_GENS)
        try_to_inc_min_seq(lrugen);
    // Read min/max seq again
    min_seq = READ_ONCE(lrugen->min_seq);
    max_seq = READ_ONCE(lrugen->max_seq);
    int oldest_gen = lru_gen_from_seq(min_seq);
    volatile unsigned int next_gen = (oldest_gen + 1) % MAX_NR_GENS;
    bpf_spin_unlock(&lrugen->lock);

    if (inc_max_seq_failed) {
        bpf_printk("cache_ext: Failed to increment max_seq\n");
    }

    int tier_threshold = get_tier_idx(lrugen);
    update_tier_selected_stat(lrugen, tier_threshold, 1);

    // Save eviction metadata for stats
    struct eviction_metadata ev_meta = {
```

```
        .curr_gen = oldest_gen,
        .next_gen = next_gen,
        .tier_threshold = tier_threshold,
    };
    set_eviction_metadata(&ev_meta);

    assert_valid_gen_0(next_gen);

    __u64 next_gen_list = mglru_lists[next_gen];
    __u64 oldest_gen_list = mglru_lists[oldest_gen];
    struct cache_ext_iterate_opts opts = {
        .continue_list = next_gen_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };


    int ret = bpf_cache_ext_list_iterate_extended(
        memcg, oldest_gen_list, mglru_iter_fn, &opts, eviction_ctx);
    if (ret < 0) {
        bpf_printk("cache_ext: Failed to iterate list\n");
        return;
    }
    struct eviction_metadata *eviction_meta = get_eviction_metadata();
    if (eviction_meta == NULL) return;
    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        min_seq = READ_ONCE(lrugen->min_seq);
        oldest_gen = lru_gen_from_seq(min_seq);
        next_gen = (oldest_gen + 1) % MAX_NR_GENS;
        __u64 next_gen_list = mglru_lists[next_gen];
        __u64 oldest_gen_list = mglru_lists[oldest_gen];
        struct cache_ext_iterate_opts opts = {
            .continue_list = next_gen_list,
            .continue_mode = CACHE_EXT_ITERATE_TAIL,
            .evict_list = CACHE_EXT_ITERATE_SELF,
            .evict_mode = CACHE_EXT_ITERATE_TAIL,
        };
        int ret = bpf_cache_ext_list_iterate_extended(
            memcg, oldest_gen_list, mglru_iter_fn, &opts, eviction_ctx);
        if (ret < 0) {
            bpf_printk("cache_ext: Failed to iterate list\n");
            return;
        }
    }
    s64 success_evicted = eviction_ctx->nr_folios_to_evict;
    s64 failed_evicted = max(0, eviction_ctx->request_nr_folios_to_evict - eviction_ctx->nr_folios_to_evict);
    __sync_fetch_and_add(&lrugen->failed_evicted, failed_evicted);
    __sync_fetch_and_add(&lrugen->success_evicted, success_evicted);
    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        bpf_printk("cache_ext: Failed to evict requested number of folios: %d/%d. Used list idx %d, list ptr: %p. Iterated %d folios",
                eviction_ctx->nr_folios_to_evict,
                eviction_ctx->request_nr_folios_to_evict,
                oldest_gen,
                oldest_gen_list,
                eviction_meta->iter_reached);
    }
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    lru_gen_add_folio(folio, is_scanning_pid());
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    folio_inc_refs(folio);
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    DEFINE_LRUGEN_void;
    // Remove tracked metadata
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata) {
        bpf_printk(
            "cache_ext: Tried to delete folio metadata but not found in map.\n");
        return;
    }
    // Add ghost entry for refault detection
    int tier = lru_tier_from_refs(atomic_long_read(&metadata->accesses));
    insert_ghost_entry_for_folio(folio, tier);

    // Update generation page count
    update_evicted_stat(lrugen, tier, 1);
    update_nr_pages_stat(lrugen, metadata->gen, -folio_nr_pages(folio));

    bpf_map_delete_elem(&folio_metadata_map, &key);
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the MGLRU policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <signal.h>
#include <string.h>
#include <stdlib.h>
#include <stdint.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "evo_policy.skel.h"
#include "dir_watcher.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
    char *cgroup_path;
    uint64_t cgroup_size;   /* accepted for harness compatibility */
};

static struct argp_option options[] = { { "watch_dir", 'w', "DIR", 0,
                      "Directory to watch" },
                    { "cgroup_path", 'c', "PATH", 0,
                      "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)" },
                    { "cgroup_size", 's', "SIZE", 0,
      "Size of the cgroup in bytes (accepted for harness compatibility)" },
                    { 0 } };

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
    case 'c':
        args->cgroup_path = arg;
        break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno)
            args->cgroup_size = 0;
        break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}
```

```
static volatile sig_atomic_t exiting;

static void sig_handler(int signo)
{
    exiting = 1;
}

int main(int argc, char **argv)
{
    int ret = 1;
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    int cgroup_fd = -1;
    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    {
        struct sigaction sa;
        memset(&sa, 0, sizeof(sa));
        sigemptyset(&sa.sa_mask);
        sa.sa_handler = sig_handler;
        if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
            perror("Failed to set up signal handling");
            return 1;
        }
    }

    // Parse command line arguments
    struct cmdline_args args = { 0 };
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, &args);

    // Validate arguments
    if (args.watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args.cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    // Does watch_dir exist?
    if (access(args.watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n",
            args.watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    char watch_dir_full_path[PATH_MAX];
    if (realpath(args.watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // TODO: Enable longer length
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    // Open cgroup directory early
    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    // Open skel
    skel = evo_policy_bpf__open();
    if (skel == NULL) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    // Set watch_dir
    skel->rodata->watch_dir_path_len = strlen(watch_dir_full_path);
    strcpy(skel->rodata->watch_dir_path, watch_dir_full_path);

    // Load programs
    {
        int scan_pids_fd = bpf_obj_get("/sys/fs/bpf/cache_ext/scan_pids");
        if (scan_pids_fd >= 0) {
            if (bpf_map__reuse_fd(skel->maps.scan_pids, scan_pids_fd))
                fprintf(stderr, "Failed to reuse scan_pids map fd\n");
            close(scan_pids_fd);
        } else {
            fprintf(stderr, "no scan_pids pin found\n");
        }
    }

    ret = evo_policy_bpf__load(skel);
    if (ret) {
        perror("Failed to load BPF skeleton");
        goto cleanup;
    }

    // Initialize inode_watchlist map
    ret = initialize_watch_dir_map(args.watch_dir,
                      bpf_map__fd(skel->maps.inode_watchlist), false);

    // Attach cache_ext_ops to the specific cgroup
    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        goto cleanup;
    }

    // Attach probes
    ret = evo_policy_bpf__attach(skel);
    if (ret) {
        perror("Failed to attach BPF programs");
        goto cleanup;
    }

    printf("evo_policy (MGLRU) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

one-seed / max-throughput (pc320): best policy #6 46ce09fd — score 540.8, 540.8 ops/s

S3-FIFO plus scan-thread detection without the harness map: a thread that adds at least 25% of all folios in an epoch of 2^11 adds is treated as the scanner. Its folios go to a dedicated SCAN list that is evicted first, and its accesses do not count as hotness. A reserve of cache/256 and a frequency cap of 3 are also used. This attempt is a re-measurement of an earlier design (#3 5397e6b9), identical apart from a comment.

Agent's title for the attempt: noise check: rerun of 5397e6b (scan detection, reserve cache/256, cap 3) unchanged except comment

Ancestor: s3fifo.c, closest seed by line similarity (89%). Lines that differ from the ancestor are highlighted.

```
// rerun of 5397e6b for noise check
// ==========================================================================
// SECTION: BPF KERNEL CODE
// ==========================================================================
// EVOLVE-BLOCK-START
// S3-FIFO: a small FIFO (~10% of the cache) admits new folios; a folio re-accessed while in
// the small queue is promoted to the main queue, otherwise evicted. The main queue is
// CLOCK-like with a frequency counter capped at 3. A ghost map remembers recently evicted
// folios so a quick re-reference is admitted straight into main.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define ENOENT      2  /* include/uapi/asm-generic/errno-base.h */
#define INT64_MAX   (9223372036854775807LL)

// Set from userspace. In terms of number of pages.
// TODO: change

//#define CACHE_SIZE (((1ull << 30) * 2) / 4096)
#define CACHE_SIZE (((1ull << 20) * 200) / 4096)
const volatile size_t cache_size = 0;

struct folio_metadata {
    s64 freq;
    u8 list; /* LIST_SMALL / LIST_MAIN / LIST_SCAN */
};

#define LIST_SMALL 0
#define LIST_MAIN 1
#define LIST_SCAN 2

/*
 * Scan-thread detection: a thread that adds a large share of all folios in an
 * epoch of 2^EPOCH_SHIFT adds is treated as the scanner. Its folios go to a
 * dedicated scan list that is evicted first, and its accesses don't count as
 * hotness.
 */
#define EPOCH_SHIFT 11
#define SCAN_SHARE_THRESH ((1 << EPOCH_SHIFT) / 4)

struct tid_stat {
    u64 epoch;
    u64 cur;
    u64 prev;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, u32);
    __type(value, struct tid_stat);
    __uint(max_entries, 8192);
} tid_stats SEC(".maps");

static u64 total_adds = 0;

struct ghost_entry {
    u64 address_space;
    u64 offset;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, u64);
    __type(value, struct folio_metadata);
    __uint(max_entries, 4000000);
} folio_metadata_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ghost_entry);
    __type(value, u8);
    //__uint(max_entries, CACHE_SIZE); // TODO: change
    __uint(map_flags, BPF_F_NO_COMMON_LRU);  // Per-CPU LRU eviction logic
} ghost_map SEC(".maps");

static u64 main_list;
static u64 small_list;
static u64 scan_list;
static s64 scan_list_size = 0;

/*
 * This is an approximate value based on what we choose to evict, not what is
 * actually evicted.
 */
static s64 small_list_size = 0;
static s64 main_list_size = 0;

static inline bool is_folio_relevant(struct folio *folio) {
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;

    return inode_in_watchlist(folio->mapping->host->i_ino);
}

static inline struct folio_metadata *get_folio_metadata(struct folio *folio) {
    u64 key = (u64)folio;
    return bpf_map_lookup_elem(&folio_metadata_map, &key);
}

static inline bool tid_is_scanner(bool count_add)
{
    u32 tid = (u32)bpf_get_current_pid_tgid();
    u64 epoch;
    if (count_add)
        epoch = __sync_fetch_and_add(&total_adds, 1) >> EPOCH_SHIFT;
    else
        epoch = total_adds >> EPOCH_SHIFT;

    struct tid_stat *st = bpf_map_lookup_elem(&tid_stats, &tid);
    if (!st) {
        if (!count_add)
            return false;
        struct tid_stat ns = { .epoch = epoch, .cur = 1, .prev = 0 };
        bpf_map_update_elem(&tid_stats, &tid, &ns, BPF_NOEXIST);
        return false;
    }

    u64 cur = st->cur, prev = st->prev;
    if (st->epoch != epoch) {
        prev = (st->epoch + 1 == epoch) ? cur : 0;
        cur = 0;
        if (count_add) {
            st->prev = prev;
            st->cur = 0;
            st->epoch = epoch;
        }
    }
    if (count_add) {
        st->cur = cur + 1;
        cur++;
    }
    return prev >= SCAN_SHARE_THRESH || cur >= SCAN_SHARE_THRESH;
}

/*
 * Check if a folio is in the ghost map and delete the ghost entry.
 * We only check if an element is in the ghost map on inserting into the cache.
 * Relies on bpf_map_delete_elem() returning -ENOENT if the element is not found.
 */
static inline bool folio_in_ghost(struct folio *folio) {
```

```
    struct ghost_entry key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    // TODO: handle non-ENOENT errors
    return bpf_map_delete_elem(&ghost_map, &key) != -ENOENT;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    main_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (main_list == 0) {
        bpf_printk("cache_ext: init: Failed to create main_list\n");
        return -1;
    }
    bpf_printk("cache_ext: Created main_list: %llu\n", main_list);

    small_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (small_list == 0) {
        bpf_printk("cache_ext: init: Failed to create small_list\n");
        return -1;
    }
    bpf_printk("cache_ext: Created small_list: %llu\n", small_list);

    scan_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (scan_list == 0) {
        bpf_printk("cache_ext: init: Failed to create scan_list\n");
        return -1;
    }

    return 0;
}

static s64 bpf_s3fifo_score_main_fn(struct cache_ext_list_node *a) {
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return INT64_MAX;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return INT64_MAX;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return INT64_MAX;
    }

    s64 freq = __sync_sub_and_fetch(&data->freq, 1);
    if (freq < 0)
        data->freq = 0;

    return freq;
}

static int bpf_s3fifo_score_small_fn(int idx, struct cache_ext_list_node *a)
{
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Move to main list if freq > 1
    if (data->freq > 1) {
        data->list = LIST_MAIN;
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Else, evict
    return CACHE_EXT_EVICT_NODE;
}

static void evict_main(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct sampling_options opts = {
        .sample_size = 10,
    };

    if (bpf_cache_ext_list_sample(memcg, main_list, bpf_s3fifo_score_main_fn, &opts,
                      eviction_ctx)) {
        bpf_printk("cache_ext: evict: Failed to sample main_list\n");
        return;
    }
    // if (__sync_sub_and_fetch(&main_list_size, eviction_ctx->nr_folios_to_evict) < 0)
    //  main_list_size = 0;
}

#define MAIN_ITER_FN(id)                                \
static int bpf_s3fifo_score_main_iter_fn_##id(int idx, struct cache_ext_list_node *a)   \
{                                                       \
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))        \
        return CACHE_EXT_CONTINUE_ITER;                     \
                                                        \
    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))       \
        return CACHE_EXT_CONTINUE_ITER;                     \
                                                        \
    struct folio_metadata *data = get_folio_metadata(a->folio);             \
    if (!data) {                                        \
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");        \
        return CACHE_EXT_CONTINUE_ITER;                     \
    }                                                   \
                                                        \
    s64 freq = __sync_sub_and_fetch(&data->freq, 1);                \
    if (freq < id) {                                    \
        /*data->freq = 0;*/                                 \
        return CACHE_EXT_EVICT_NODE;                        \
    }                                                   \
                                                        \
    return CACHE_EXT_CONTINUE_ITER;                         \
}

MAIN_ITER_FN(0)
MAIN_ITER_FN(1)
MAIN_ITER_FN(2)
MAIN_ITER_FN(3)

static void evict_main_iter(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_0, &opts,
                        eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_1, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_2, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_3, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
```

```
    }
}

static void evict_small(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 1, move to main list, otherwise evict.
     * (When evicting, move to tail in the meantime).
     *
     * Use the iterate interface.
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = main_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, small_list, bpf_s3fifo_score_small_fn, &opts,
                        eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate small_list\n");
        return;
    }

    if (__sync_fetch_and_sub(&small_list_size, opts.nr_folios_continue) < 0)
        small_list_size = 0;

    if (__sync_fetch_and_add(&main_list_size, opts.nr_folios_continue) < 0)
        main_list_size = opts.nr_folios_continue;
}

static int bpf_score_scan_fn(int idx, struct cache_ext_list_node *a)
{
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return CACHE_EXT_CONTINUE_ITER;
    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data)
        return CACHE_EXT_CONTINUE_ITER;

    // Touched by a non-scanner thread: it's GET-hot, promote to main
    if (data->freq > 0) {
        data->list = LIST_MAIN;
        return CACHE_EXT_CONTINUE_ITER;
    }
    return CACHE_EXT_EVICT_NODE;
}

static void evict_scan(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    struct cache_ext_iterate_opts opts = {
        .continue_list = main_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, scan_list, bpf_score_scan_fn, &opts,
                        eviction_ctx) < 0)
        return;

    if (__sync_fetch_and_sub(&scan_list_size, opts.nr_folios_continue) < 0)
        scan_list_size = 0;
    __sync_fetch_and_add(&main_list_size, opts.nr_folios_continue);
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    // bpf_printk("cache_ext: evict_folios: main_list_size: %lld, small_list_size: %lld, cache_size: %lld\n",
    //     main_list_size, small_list_size, cache_size);
    if (scan_list_size > (s64)(cache_size / 256)) {
        evict_scan(eviction_ctx, memcg);
        if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
            return;
    }

    if (small_list_size >= cache_size / 15 || main_list_size <= 2 * small_list_size)
        evict_small(eviction_ctx, memcg);
    else
        evict_main_iter(eviction_ctx, memcg);
}
```

```
void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    struct folio_metadata *data = get_folio_metadata(folio);
    if (!data)
        return;

    // The scanner's own accesses are not a hotness signal
    if (tid_is_scanner(false))
        return;

    // Cap frequency at 3
    if (__sync_add_and_fetch(&data->freq, 1) > 3)
        data->freq = 3;
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
    u64 key = (u64)folio;
    u8 ghost_val = 0;

    // if (bpf_cache_ext_list_del(folio)) {
    //  bpf_printk("cache_ext: Failed to delete folio from sampling_list\n");
    //  return;
    // }

    struct ghost_entry ghost_key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };

    // Don't return early, we want to delete the folio metadata regardless
    if (bpf_map_update_elem(&ghost_map, &ghost_key, &ghost_val, BPF_ANY))
        bpf_printk("cache_ext: evicted: Failed to add to ghost_map\n");

    struct folio_metadata *data = get_folio_metadata(folio);
    if (!data) {
        //bpf_printk("cache_ext: evicted: Failed to get metadata\n");
        return;
    }

    if (data->list == LIST_MAIN)
        __sync_fetch_and_sub(&main_list_size, 1);
    else if (data->list == LIST_SCAN)
        __sync_fetch_and_sub(&scan_list_size, 1);
    else
        __sync_fetch_and_sub(&small_list_size, 1);

    bpf_map_delete_elem(&folio_metadata_map, &key);

    // if (bpf_map_delete_elem(&folio_metadata_map, &key))
    //  bpf_printk("cache_ext: evicted: Failed to delete metadata\n");
}

/*
 * If folio is in the ghost map, add to tail of main list, otherwise add to tail
 * of small list.
 */
void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    u64 key = (u64)folio;
    struct folio_metadata new_meta = {
        .freq = 0,
    };

    u64 list_to_add;
    bool scanner = tid_is_scanner(true);
    if (scanner) {
        list_to_add = scan_list;
        new_meta.list = LIST_SCAN;
          sync fetch and add(&scan list_size, 1);
    } else if (folio_in_ghost(folio)) {
        list_to_add = main_list;
        new_meta.list = LIST_MAIN;
        __sync_fetch_and_add(&main_list_size, 1);
    } else {
        list_to_add = small_list;
        new_meta.list = LIST_SMALL;
        __sync_fetch_and_add(&small_list_size, 1);
    }

    if (bpf_cache_ext_list_add_tail(list_to_add, folio)) {
        // TODO: add back to ghost_map?
        bpf_printk("cache_ext: added: Failed to add folio to main_list\n");
```

```
        return;
    }

    if (bpf_map_update_elem(&folio_metadata_map, &key, &new_meta, BPF_ANY)) {
        // TODO: add back to ghost_map? + error check delete call?
        bpf_cache_ext_list_del(folio);
        bpf_printk("cache_ext: added: Failed to create folio metadata\n");
        return;
    }
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the S3-FIFO policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <stdlib.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "dir_watcher.h"
#include "evo_policy.skel.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_size <size> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
        uint64_t cgroup_size;
        char *cgroup_path;
};

static struct argp_option options[] = {
    { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
        {"cgroup_size", 's', "SIZE", 0, "Size of the cgroup"},
        {"cgroup_path", 'c', "PATH", 0, "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)"},
    { 0 },
};

static const uint64_t page_size = 4096;

static volatile sig_atomic_t exiting;

static void sig_handler(int signo) {
    exiting = 1;
}

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
        case 's':
                // TODO: move this to parse_args()
                errno = 0;
                args->cgroup_size = strtoull(arg, NULL, 10);
                if (errno)
                        args->cgroup_size = 0;

                break;
        case 'c':
                args->cgroup_path = arg;
                break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static int parse_args(int argc, char **argv, struct cmdline_args *args) {
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, args);

    if (args->watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args->cgroup_size == 0) {
            fprintf(stderr, "Invalid cgroup size\n");
            return 1;
    }

    if (args->cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    return 0;
}

/*
 * Validate watch_dir
 *
 * watch_dir_full_path must be able to hold PATH_MAX bytes.
 */
static int validate_watch_dir(const char *watch_dir, char *watch_dir_full_path) {
    // Does watch_dir exist?
    if (access(watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n", watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    if (realpath(watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // BPF policy restriction
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    struct sigaction sa;
    char watch_dir_path[PATH_MAX];
    int cgroup_fd = -1;
    int ret = 1;

    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    if (parse_args(argc, argv, &args))
        return 1;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;

    // Install signal handler
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }

    if (validate_watch_dir(args.watch_dir, watch_dir_path))
        return 1;

    // Open cgroup directory early
    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }
```

```
    skel = evo_policy_bpf__open();
    if (!skel) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    // Set cache size in terms of number of pages. Assumes uniform page size.
    skel->rodata->cache_size = args.cgroup_size / page_size;
    fprintf(stderr, "Cgroup size: %lu bytes\n", args.cgroup_size);
    fprintf(stderr, "Cache size: %lu pages\n", skel->rodata->cache_size);

    // Resize ghost_map
    if (bpf_map__set_max_entries(skel->maps.ghost_map, skel->rodata->cache_size)) {
        perror("Failed to resize ghost_map");
        ret = 1;
        goto cleanup;
    }

    // Set watch_dir
    watch_dir_path_len_map(skel) = strlen(watch_dir_path);
    strcpy(watch_dir_path_map(skel), watch_dir_path);

    if (evo_policy_bpf__load(skel)) {
        perror("Failed to load BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    if (initialize_watch_dir_map(watch_dir_path, bpf_map__fd(inode_watchlist_map(skel)), true)) {
        perror("Failed to initialize watch_dir map");
        ret = 1;
        goto cleanup;
    }

    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        ret = 1;
        goto cleanup;
    }

    // This is necessary for the dir_watcher functionality
    if (evo_policy_bpf__attach(skel)) {
        perror("Failed to attach BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    printf("evo_policy (S3-FIFO) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;
cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```